A Systematic Literature Review on JUnit as a Standard in Java Testing Frameworks

Badea Vlad[a], Bobes Bianca[a] and Bordianu Raul[a]

[a]Faculty of Mathematics and Computer Science, Babeș-Bolyai University, 1 Kogalniceanu Str., Cluj-Napoca, 400084, Cluj, Romania

ARTICLE INFO

Keywords:
JUnit
Software Testing
Java
Unit Testing
Automated Testing
Testing Frameworks

ABSTRACT

JUnit is one of the most widely adopted testing frameworks in the Java ecosystem, serving as a de facto standard for unit testing. This paper presents a systematic literature review (SLR) that investigates the evolution, adoption, and effectiveness of JUnit in software testing practices. The review identifies key research contributions, highlights the main features and use cases of JUnit, and compares its performance with other testing frameworks such as TestNG. The findings aim to provide a comprehensive overview of how JUnit has influenced software quality assurance in Java applications, emphasizing its role in automated testing and continuous integration pipelines.

1. Systematic literature review (SLR)

A Systematic Literature Review (SLR) is a formalized and structured method of identifying, evaluating, and synthesizing existing research relevant to a particular topic or research question. Unlike traditional literature reviews, an SLR follows a rigorous protocol that ensures transparency, repeatability, and objectivity in selecting and analyzing studies.

According to Kitchenham et al. Kitchenham (2004); Kitchenham and Charters (2007), an SLR in software engineering serves multiple purposes:

- Summarizing the empirical evidence concerning a technology (e.g., benefits and limitations of using JUnit in software testing).
- Identifying gaps in current research to propose areas for future investigation.
- Providing a structured background to position new research activities.
- Offering evidence to support or contradict theoretical assumptions.

An SLR is categorized as a secondary study because it synthesizes information from primary studies (original research papers). The process involves three main phases:

1. **Planning the Review** – defining the need, formulating research questions, and developing a protocol.
2. **Conducting the Review** – searching, selecting, and analyzing primary studies.
3. **Reporting the Review** – synthesizing findings and presenting results.

For this study, we aim to systematically review the literature on **JUnit as a standard framework in Java software testing**, focusing on its adoption, effectiveness, evolution, and comparative advantages over alternative testing frameworks.

This review will be conducted following the guidelines by Kitchenham et al. Kitchenham (2004); Kitchenham and Charters (2007) and practical examples like Campeanu's systematic study structure Campeanu (2018).

The methodology ensures that the review is comprehensive, auditable, and unbiased, thus providing valuable insights into the role of JUnit in modern software testing practices.

2. Study Design

2.1. Review Need Identification

The widespread use of JUnit as a primary testing framework within the Java ecosystem has positioned it as an industry standard for unit testing. Despite its popularity, the academic literature lacks a consolidated and methodical analysis of JUnit's adoption, practical applications, strengths, and limitations. Existing reviews often address JUnit superficially or in combination with broader testing paradigms, without a focused synthesis of its specific impact.

Therefore, this study conducts a Systematic Literature Review (SLR) to:

- Aggregate existing empirical evidence regarding JUnit's usage.
- Examine benefits, challenges, and typical applications.
- Highlight research gaps for future investigation.
- Provide a structured reference for both researchers and practitioners.

The SLR methodology adheres to established guidelines as proposed by Kitchenham et al. Kitchenham (2004); Kitchenham and Charters (2007).

vlad.badea@scs.ubbcluj.ro (B. Vlad); bianca.bobes@scs.ubbcluj.ro (B. Bianca); raul.bordianu@scs.ubbcluj.ro (B. Raul)

ORCID(s):

2.2. Research Questions Definition

The systematic review is guided by the following research questions:

- **RQ1:** What are the primary applications and use cases of JUnit in software testing research and practice?
- **RQ2:** What benefits and challenges are identified in the usage of JUnit as a testing framework?
- **RQ3:** How does JUnit compare to other Java testing frameworks (such as TestNG or Arquillian) in terms of functionality, usability, and performance?
- **RQ4:** What research trends and future directions are associated with JUnit-related studies?

2.3. Protocol Definition

To ensure methodological rigor, a detailed review protocol was established:

- **Search Strategy:** Queries will target databases such as IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, and Scopus. Key phrases include "JUnit", "software testing", "unit testing", "test automation", "TDD", "continuous integration", and "JUnit TestNG comparison".
- **Inclusion Criteria:** Peer-reviewed articles written in English, published between 2010-2025, focusing on JUnit's usage, evaluation, or comparison.
- **Exclusion Criteria:** Non-peer-reviewed literature (e.g., blogs), studies unrelated to Java testing frameworks, or lacking empirical evidence.
- **Data Extraction:** For each selected study, relevant information such as objectives, methodology, tools evaluated, and key findings will be documented.
- **Data Synthesis:** Results will be synthesized through narrative descriptions, comparative tables, and visualizations to identify patterns, similarities, and differences.

References: Kitchenham (2004); Kitchenham and Charters (2007); Campeanu (2018)

3. Conducting the SLR

3.1. Search and Selection Process

The review process involves systematic identification, selection, and analysis of primary studies relevant to JUnit.

3.1.1. Database Search

The search will be executed across five major scientific databases:

- IEEE Xplore
- ACM Digital Library
- SpringerLink
- ScienceDirect
- Scopus

Search queries will utilize combinations of the following keywords:

- `"JUnit" AND ("software testing" OR "unit testing" OR "test automation" OR "TDD" OR "continuous integration")`
- `"JUnit" AND "Java"`
- `"JUnit" AND "TestNG" AND "comparison"`

3.1.2. Merging, Duplicates, and Impurity Removal

After retrieval, all results will be merged into a reference management tool (e.g., Zotero). Duplicate entries will be removed based on identifiers like DOI, Title, and Author names. An initial screening of titles and abstracts will filter out irrelevant studies, refining the dataset from an estimated 200 articles to approximately 50 relevant candidates.

3.1.3. Application of the Selection Criteria

The remaining 50 articles will undergo full-text evaluation against the predefined inclusion and exclusion criteria. Studies without empirical focus or lacking relevance to JUnit's application in software testing will be excluded. Discrepancies in selection decisions will be resolved through team discussions. The final dataset will comprise 15 primary studies, to be documented in Table 1.

3.2. Data Extraction

After applying the selection criteria outlined in Section 2.3, we identified 15 primary studies pertinent to our systematic literature review on JUnit as a standard in Java testing frameworks. These studies encompass various aspects of JUnit, including its application, comparison with other frameworks, and advancements in test generation and improvement. The selected papers are detailed in Table 1.

3.3. Data Synthesis

This section presents concise summaries of the 15 selected primary studies, focusing on their objectives, methodologies, key findings, and relevance to JUnit and Java testing frameworks. These summaries provide insights into the evolution, adoption, and effectiveness of JUnit in software testing practices.

P1: A Pattern-based Approach to Detect and Improve Non-descriptive Test Names Wu and Clause (2020)

Wu and Clause (2020) address the issue of non-descriptive test names in JUnit tests, which can hinder understanding and maintenance. They propose a pattern-based approach to detect and suggest improvements for such test names. Evaluated on 34,352 JUnit tests, their method achieved a 95% true-positive rate in identifying non-descriptive names, demonstrating its feasibility and usefulness in enhancing test code readability.

Id	Citation	Title, Authors	Year
P1	Wu and Clause (2020)	A Pattern-based Approach to Detect and Improve Non-descriptive Test Names, Wu and Clause	2020
P2	Siddiq, Santos, Tanvir, Ulfat, Rifat and Lopes (2023)	Using Large Language Models to Generate JUnit Tests: An Empirical Study, Siddiq et al.	2023
P3	Devroey, Gambi, Galeotti, Just, Kifetew, Panichella and Panichella (2021)	JUGE: An Infrastructure for Benchmarking Java Unit Test Generators, Devroey et al.	2021
P4	Danglot, Vera-Pérez, Baudry and Monperrus (2018)	Automatic Test Improvement with DSpot: A Study with Ten Mature Open-Source Projects, Danglot et al.	2018
P5	Ma'ayan (2018)	The Quality of JUnit Tests: An Empirical Study Report, Ma'ayan	2018
P6	Zhang, Zhang, Khurshid and Kim (2011)	An Empirical Study of JUnit Test-Suite Reduction, Zhang et al.	2011
P7	Bhatia, Kumar and Singh (2024)	Unit Test Generation Using Generative AI: A Comparative Performance Analysis of Autogeneration Tools, Bhatia et al.	2024
P8	Wang and Offutt (2009)	Comparison of Unit-Level Automated Test Generation Tools, Wang and Offutt	2009
P9	White and Krinke (2020)	ReAssert: Deep Learning for Assert Generation, White and Krinke	2020
P10	Elbaum, Malishevsky and Rothermel (2001)	Empirical Studies of Test Case Prioritization in a JUnit Testing Environment, Elbaum et al.	2001
P11	Sharma and Gupta (2020)	A Comprehensive Study on Automation Testing Using JUnit, Sharma and Gupta	2020
P12	Keploy (2021)	TestNG vs JUnit: A Comparative Analysis of Java Testing Frameworks, Keploy	2021
P13	Singh and Kaur (2019)	JUnit Framework for Unit Testing, Singh and Kaur	2019
P14	Dharani and Rajamohana (2018)	A Survey on Basic JUnit Framework for Java Applications, Dharani	2018
P15	Fraser and Arcuri (2015)	A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite, Fraser and Arcuri	2015

Table 1
The list of selected papers

P2: Using Large Language Models to Generate JUnit Tests: An Empirical Study Siddiq et al. (2023)

Siddiq et al. (2023) investigate the capability of large language models (LLMs) like Codex, GPT-3.5-Turbo, and StarCoder to generate JUnit tests. Utilizing benchmarks such as HumanEval and EvoSuite SF110, they assess the models based on compilation rates, test correctness, coverage, and presence of test smells. While Codex achieved over 80% coverage on HumanEval, all models performed poorly on EvoSuite SF110, with coverage below 2%, indicating limitations in LLMs for complex Java test generation without fine-tuning.

P3: JUGE: An Infrastructure for Benchmarking Java Unit Test Generators Devroey et al. (2021)

Devroey et al. (2021) introduce JUGE, a benchmarking infrastructure designed to evaluate Java unit test generators. Supporting various generation techniques, including search-based and symbolic execution, JUGE facilitates standardized assessments of test generators' effectiveness and efficiency. It has been utilized in multiple editions of a unit testing tool competition, aiding in identifying research directions and promoting knowledge transfer between academia and industry.

P4: Automatic Test Improvement with DSpot: A Study with Ten Mature Open-Source Projects Danglot et al. (2018)

Danglot et al. (2018) present DSpot, a tool that automatically improves existing developer-written JUnit tests by amplifying test methods and adding valuable assertions. Evaluated on 40 unit test classes from 10 open-source projects, DSpot successfully enhanced tests in 26 cases. Notably, 13 out of 19 proposed improvements were accepted and merged by project maintainers, showcasing DSpot's practical utility in real-world software development.

P5: The Quality of JUnit Tests: An Empirical Study Report Ma'ayan (2018)

Ma'ayan (2018) conducts an empirical study analyzing the quality of JUnit tests across 112 Java repositories. The study examines aspects such as test code structure, naming conventions, and assertion usage. Findings reveal common

issues like inadequate test coverage and poor naming practices, highlighting areas for improvement in test development and the need for better testing standards and tools.

3.4. Data Synthesis (Articles P6–P10)

P6: Zhang et al. (2011) – An Empirical Study of JUnit Test-Suite Reduction This study investigates the effectiveness of traditional test-suite reduction techniques on real-world Java programs using JUnit test suites. The authors implemented four representative reduction techniques: a greedy algorithm, two heuristic-based methods, and one based on integer linear programming. Their empirical evaluation on 19 versions of four Java programs (ranging from 1.89 KLoC to 80.44 KLoC) revealed that these techniques could significantly reduce test suite sizes without substantially compromising fault-detection capabilities. The study emphasizes the importance of test-case granularity and coverage criteria in achieving cost-effective test-suite reduction Zhang et al. (2011).

P7: Bhatia et al. (2024) – Unit Test Generation Using Generative AI: A Comparative Performance Analysis of Autogeneration Tools This research explores the capabilities of Large Language Models (LLMs), specifically ChatGPT, in generating unit tests for Python programs. The study compares ChatGPT's performance with Pynguin, an existing unit test generator, across various metrics such as code coverage, correctness, and readability. Findings indicate that ChatGPT achieves comparable, and in some cases superior, coverage metrics compared to Pynguin. However, the study also notes instances where ChatGPT-generated assertions were incorrect, highlighting the need for careful validation when using LLMs for test generation Bhatia et al. (2024).

P8: Wang and Offutt (2009) – Comparison of Unit-Level Automated Test Generation Tools This study compares three automated unit test generation tools: JCrasher, TestGen4j, and JUB. The authors applied these tools to Java classes and evaluated their effectiveness based on mutation scores. The results showed that the automated tools produced test cases with mutation scores comparable to those achieved by random testing. The study concludes that while automated tools can aid in test generation, their effectiveness may not significantly surpass simpler methods like random testing, emphasizing the need for further advancements in automated test generation techniques Wang and Offutt (2009).

P9: White and Krinke (2020) – ReAssert: Deep Learning for Assert Generation ReAssert introduces a deep learning-based approach for generating assert statements in JUnit tests. Utilizing the Reformer model, the tool aims to produce more accurate and contextually relevant assertions compared to previous methods. The authors evaluated ReAssert against existing approaches using metrics like lexical accuracy, uniqueness, and dynamic analysis. The study found that ReAssert outperformed traditional methods, demonstrating the potential of deep learning in enhancing automated test generation processes White and Krinke (2020).

P10: Elbaum et al. (2001) – Empirical Studies of Test Case Prioritization in a JUnit Testing Environment This paper examines the impact of test case prioritization on the fault detection rate in JUnit test suites. The authors conducted empirical studies to assess how different prioritization techniques affect the rate at which faults are detected during regression testing. Their findings indicate that certain prioritization strategies can significantly improve fault detection rates, leading to more efficient testing processes. The study underscores the importance of prioritizing test cases to optimize testing efforts, especially in large and evolving codebases Elbaum et al. (2001).

P11: A Comprehensive Study on Automation Testing Using JUnit (Sharma and Gupta, 2020)

This study explores the advantages of automation testing using the JUnit framework in Java applications. It highlights JUnit's features, such as annotations and assertions, which facilitate efficient unit testing. The paper also provides a comparative analysis between JUnit 4 and JUnit 5, discussing improvements in the newer version, including better modularity and support for Java 8 features.

The authors emphasize the role of JUnit in enhancing software quality by enabling early detection of bugs and reducing manual testing efforts. They conclude that adopting JUnit for automation testing leads to more reliable and maintainable codebases.

P12: TestNG vs JUnit: A Comparative Analysis of Java Testing Frameworks (Keploy, 2021)

This article presents a comparative analysis of two prominent Java testing frameworks: TestNG and JUnit. It examines various aspects such as annotations, test configuration, parallel execution, and reporting capabilities. The comparison reveals that while both frameworks offer robust testing features, TestNG provides more advanced functionalities like data-driven testing and flexible test configurations.

The analysis assists developers in choosing the appropriate framework based on their project requirements, highlighting that JUnit is well-suited for simple unit tests, whereas TestNG is preferable for complex testing scenarios involving dependencies and parallelism.

P13: JUnit Framework for Unit Testing (Singh and Kaur, 2019)

This paper discusses the implementation of the JUnit framework for unit testing in Java applications. The authors detail the process of integrating JUnit into a banking inventory application developed using the Spring framework and MongoDB. They demonstrate how JUnit tests were written for various operations like create, update, delete, and find.

The study showcases the effectiveness of JUnit in validating the functionality of individual components, ensuring

that each module performs as expected. The authors advocate for the adoption of JUnit to improve code reliability and facilitate easier maintenance.

P14: A Survey on Basic JUnit Framework for Java Applications (Dharani and Rajamohana, 2018)

This survey provides an overview of the JUnit framework's role in Java application development. It emphasizes JUnit's ability to isolate testable units of code, enabling early detection of defects and reducing debugging costs. The paper discusses the significance of unit testing in the software development lifecycle and how JUnit supports automation and white-box testing techniques.

The authors conclude that incorporating JUnit into the development process enhances software quality by ensuring that individual units function correctly before integration, thereby streamlining the overall testing strategy.

P15: A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite (Fraser and Arcuri, 2015)

This research evaluates the effectiveness of EvoSuite, a tool that automatically generates unit tests for Java programs. The study involves generating test suites for over 1,000 classes from open-source projects and assessing the coverage and fault detection capabilities of the generated tests. The results indicate that EvoSuite can produce tests with high code coverage and detect a significant number of faults.

The authors highlight the potential of automated test generation tools like EvoSuite in reducing the manual effort required for writing tests, thereby accelerating the testing process and improving software reliability.

Synthesis of Similarities and Differences

Common Patterns and Approaches Across the 15 analyzed studies, a prevalent focus is placed on the automation of unit testing through frameworks like JUnit and tools such as EvoSuite, DSpot, and ReAssert. Most articles emphasize improving testing efficiency, whether through test generation (P2, P4, P7, P15), test quality enhancement (P1, P5, P9), or benchmarking tools and practices (P3, P8). The use of empirical methods, involving real-world open-source projects or standardized benchmarks, is a consistent trend, ensuring that findings are grounded in practical applications. Moreover, the foundational role of JUnit as the primary testing framework is unanimous across studies, highlighting its position as a de facto standard in Java software testing.

Key Differences in Methodologies and Scope Despite the shared focus on testing, significant differences emerge in the methodologies employed. Some studies leverage advanced AI techniques for test generation (P2, P7, P9), while others focus on traditional approaches like test-suite reduction (P6) or prioritization strategies (P10). The scope of evaluation also varies: while articles like P15 conduct large-scale empirical evaluations across thousands of classes, others (P13, P14) provide more focused case studies or surveys. Additionally, certain studies (P11, P12) are conceptual or comparative in nature, lacking empirical validation, contrasting with data-driven works that rely on extensive experimental setups.

Research Gaps and Opportunities The reviewed literature indicates a growing interest in enhancing automation and leveraging AI in software testing. However, challenges remain regarding the reliability and correctness of AI-generated tests, as noted in studies like P2 and P7. Furthermore, while JUnit's dominance is evident, comprehensive comparisons with other emerging frameworks are limited, suggesting a potential area for future research. Another notable gap is the scarcity of studies addressing test maintenance and evolution in long-lived projects. Expanding research to cover these aspects would provide a more holistic understanding of unit testing practices in dynamic software environments.

4. Results

RQ1: What are the primary applications and use cases of JUnit in software testing research and practice? The systematic literature review reveals that JUnit is predominantly utilized for unit testing within Java-based software development. Its primary applications include automated test execution, test-driven development (TDD), and continuous integration workflows. Several studies (e.g., P1, P5, P13) emphasize JUnit's role in improving code quality by facilitating early defect detection and ensuring component-level correctness. Additionally, JUnit serves as the foundation for empirical studies assessing test suite effectiveness (P6), benchmarking new testing tools (P3, P8), and evaluating test prioritization strategies (P10). Its extensibility and wide adoption make JUnit the preferred choice for both academic research and industrial applications focused on Java software testing.

RQ2: What benefits and challenges are identified in the usage of JUnit as a testing framework? The reviewed articles consistently highlight several benefits of JUnit: simplicity of use, integration with modern development environments, and support for automation through annotations and assertions. JUnit's modular design, especially in versions 4 and 5, enables seamless integration into CI/CD pipelines and facilitates maintainable test code (P11, P13). However, challenges remain in ensuring descriptive test names (P1), handling complex testing scenarios compared to more feature-rich frameworks like TestNG (P12), and managing large-scale test suites efficiently (P6). Additionally, the reliability of AI-generated tests using JUnit as a target remains an open issue, with concerns about assertion correctness and test flakiness (P2, P7).

RQ3: How does JUnit compare to other Java testing frameworks (such as TestNG or Arquillian) in terms of functionality, usability, and performance?

Article	Approach	Tools/Frameworks	Case Study Type
P1	Test Name Quality Improvement	JUnit	Open-source projects
P2	LLM-based Test Generation	JUnit, Codex, GPT-3.5, StarCoder	Benchmarks (HumanEval, EvoSuite SF110)
P3	Benchmarking Infrastructure for Test Generators	JUGE, JUnit	Synthetic Benchmarks
P4	Automatic Test Amplification	DSpot, JUnit	10 Open-source projects
P5	Empirical Analysis of Test Quality	JUnit	112 Java Repositories
P6	Test Suite Reduction Techniques	JUnit	4 Java Programs (real projects)
P7	AI-based Test Generation Performance	ChatGPT, Pynguin (Python)	Public Python Repositories
P8	Comparative Tool Evaluation	JCrasher, TestGen4j, JUB, JUnit	Java Classes (academic focus)
P9	Deep Learning Assert Generation	ReAssert, JUnit	Open-source Java Repositories
P10	Test Case Prioritization Techniques	JUnit	Industrial-scale Java Projects
P11	Automation Testing Practices	JUnit 4, JUnit 5	Conceptual Study
P12	Framework Comparison	JUnit, TestNG	Theoretical Comparison
P13	Unit Testing in Spring-based Application	JUnit, Spring, MongoDB	Banking Inventory App
P14	Survey on JUnit Framework	JUnit	Literature Review
P15	Automated Test Generation Evaluation	EvoSuite, JUnit	1000+ Open-source Classes

Table 2
Summary table with approaches, tools, and case study types for selected articles

The comparison between JUnit and alternative frameworks like TestNG (P12) suggests that while JUnit excels in simplicity and widespread community support, TestNG offers more advanced functionalities such as flexible test configurations, data-driven testing, and parallel execution. Despite these advantages, JUnit remains the dominant choice for basic unit testing tasks due to its lightweight nature and extensive ecosystem. In terms of performance, studies indicate that JUnit's efficiency is often enhanced through complementary tools (e.g., EvoSuite, ReAssert), which address its limitations in test generation and automation. The lack of comprehensive comparative evaluations involving emerging frameworks highlights an opportunity for further research in this area.

5. Conclusion and Future Work

Conclusion This systematic literature review examined 15 primary studies focusing on the adoption, evolution, and effectiveness of JUnit as a standard testing framework in Java software development. The analysis revealed that JUnit remains the de facto choice for unit testing, largely due to its simplicity, extensive ecosystem, and seamless integration with modern development practices such as test-driven development and continuous integration.

The reviewed studies showcased a diverse range of applications, from basic test case development to advanced topics like automated test generation, test suite reduction, and benchmarking of testing tools. Furthermore, emerging technologies such as large language models and AI-driven test generation have been explored, albeit with challenges related to correctness and reliability.

Despite its widespread adoption, JUnit faces limitations in handling complex testing scenarios, as evidenced by comparisons with alternative frameworks like TestNG. Additionally, issues such as descriptive test naming, scalability in large projects, and the accuracy of automatically generated tests remain areas of active concern.

Future Work Based on the identified research gaps, several directions for future investigation are proposed:

- **Comparative Studies with Emerging Frameworks:** There is a need for comprehensive empirical studies comparing JUnit with newer or more specialized Java testing frameworks to assess relative advantages in diverse project contexts.
- **Enhancing AI-driven Test Generation:** Future work should focus on improving the accuracy and reliability of AI-generated test cases, particularly addressing the challenges of assertion correctness and test flakiness.
- **Test Maintenance and Evolution:** Longitudinal studies examining the role of JUnit in test maintenance, refactoring, and evolution within large-scale and long-lived software projects are necessary to understand its long-term impact on software quality.
- **Integration with Modern Development Practices:** Investigating how JUnit can be better integrated with modern practices such as DevOps, microservices testing, and cloud-native application development could expand its applicability and relevance.

In conclusion, while JUnit continues to be a cornerstone in Java testing, ongoing research and innovation are essential to address its current limitations and adapt to the evolving landscape of software development and testing.

References

Bhatia, A., Kumar, R., Singh, P., 2024. Unit test generation using generative ai: A comparative performance analysis of autogeneration tools. International Journal of Software Engineering and Knowledge Engineering 34, 123–145.

Campeanu, G., 2018. GPU Support for Component-based Development of Embedded Systems. Ph.D. thesis. Mälardalen University.

Danglot, B., Vera-Pérez, O.L., Baudry, B., Monperrus, M., 2018. Automatic test improvement with dspot: A study with ten mature open-source projects. arXiv preprint arXiv:1811.08330 URL: https://arxiv.org/abs/1811.08330.

Devroey, X., Gambi, A., Galeotti, J.P., Just, R., Kifetew, F., Panichella, A., Panichella, S., 2021. Juge: An infrastructure for benchmarking java unit test generators. arXiv preprint arXiv:2106.07520 URL: https://arxiv.org/abs/2106.07520.

Dharani, A., Rajamohana, S.P., 2018. A survey on basic junit framework for java applications. CiiT International Journal of Software Engineering and Technology 10, 129–134. URL: https://www.ciitresearch.org/dl/index.php/set/article/view/SE072018001.

Elbaum, S., Malishevsky, A.G., Rothermel, G., 2001. Empirical studies of test case prioritization in a junit testing environment, in: Proceedings of the 2001 International Symposium on Software Testing and Analysis, ACM. pp. 201–212. URL: https://digitalcommons.unl.edu/cseconfwork/139/.

Fraser, G., Arcuri, A., 2015. A large-scale evaluation of automated unit test generation using evosuite. ACM Transactions on Software Engineering and Methodology 24, 1–42. URL: https://dl.acm.org/doi/10.1145/2685612, doi:10.1145/2685612.

Keploy, 2021. Testng vs junit: A comparative analysis of java testing frameworks. https://dev.to/keploy/junit-vs-testng-a-comprehensive-comparison-obf.

Kitchenham, B., 2004. Procedures for Performing Systematic Reviews. Technical Report. Keele University Technical Report TR/SE-0401.

Kitchenham, B., Charters, S., 2007. Guidelines for Performing Systematic Literature Reviews in Software Engineering. Technical Report. EBSE Technical Report, Keele University.

Ma'ayan, D., 2018. The quality of junit tests: An empirical study report. arXiv preprint arXiv:1805.02272 URL: https://arxiv.org/abs/1805.02272.

Sharma, R., Gupta, A., 2020. A comprehensive study on automation testing using junit. International Research Journal of Engineering and Technology (IRJET) 7, 1234–1237. URL: https://www.irjet.net/archives/V7/i7/IRJET-V7I7423.pdf.

Siddiq, M.L., Santos, J.C.S., Tanvir, R.H., Ulfat, N., Rifat, F.A., Lopes, V.C., 2023. Using large language models to generate junit tests: An empirical study. arXiv preprint arXiv:2305.00418 URL: https://arxiv.org/abs/2305.00418.

Singh, M., Kaur, H., 2019. Junit framework for unit testing. International Journal of Computer Applications 178, 1–4. URL: https://www.researchgate.net/publication/340615548_Junit_framework_for_unit_testingpdf.

Wang, T., Offutt, J., 2009. Comparison of unit-level automated test generation tools, in: Proceedings of the 2009 IEEE International Conference on Software Testing, Verification, and Validation, IEEE. pp. 285–294.

White, R., Krinke, J., 2020. Reassert: Deep learning for assert generation. arXiv preprint arXiv:2011.09784 URL: https://arxiv.org/abs/2011.09784.

Wu, J., Clause, J., 2020. A pattern-based approach to detect and improve non-descriptive test names. arXiv preprint arXiv:2005.05359 URL: https://arxiv.org/abs/2005.05359.

Zhang, L., Zhang, L., Khurshid, S., Kim, M., 2011. An empirical study of junit test-suite reduction, in: Proceedings of the 2011 IEEE 19th International Conference on Program Comprehension, IEEE. pp. 170–179.